

Safe Area & Responsive Layout

Modern devices have notches, status bars, and edge-to-edge displays, so mobile UIs must account for system insets. In Flutter, wrap top-level widgets in a `SafeArea` or otherwise apply padding so content stays “within the safe area” [2†L62-L68] . This ensures no content (like the toolbar) is drawn under the status bar or notch. On Android 15+ (API 35+), edge-to-edge is automatic: by default the app’s content may extend into cutouts unless you add padding. Google advises detecting cutout locations (especially in landscape where a notch can be on the side) and adding appropriate padding [5†L19-L23] . In practice, use Flutter’s `SafeArea` or `MediaQuery.padding` and verify that in landscape orientation your UI shifts below the system bar.

Beyond safe-area, use responsive design to adapt to screen size/orientation. A common pattern is “**Mobile-first**”: design one layout for phones (portrait) and adapt it for larger/tablet screens or landscape. For example, detect the screen width and orientation via `MediaQuery` or `LayoutBuilder` and switch layouts. On narrow screens, put navigation in a bottom bar; on wide screens use a navigation rail and split-pane layout [29†L1-L4] [29†L7-L10] . As one Flutter guide shows, you can do something like:

```
final screenSize = getScreenSize(context);
return Scaffold(
  // Phone: show bottom nav bar; tablet: hide it
  bottomNavigationBar: switch(screenSize) {
    ScreenSize.normal || ScreenSize.small => BottomNavBar(),
    _ => null,
  },
  body: switch(screenSize) {
    // Tablet/desktop: split view with NavigationRail and two columns
    ScreenSize.large || ScreenSize.extraLarge => Row(
      children: [NavigationRail(...), Expanded(child: MainPane),
Expanded(child: DetailPane)]
    ),
    // Phones: single-column view
    _ => MainPane(),
  },
);
```

This kind of layout switching (as shown in [29]) avoids overlapping system UI and makes the UI usable in both portrait and landscape. In summary: **fix any toolbar overlap by enclosing it in a `SafeArea` (or `Scaffold.appBar`) and test in landscape**, and implement adaptive breakpoints so your UI reorganizes rather than stretches.

Storage & File-Picker Integration

To import images or motifs, the app should use a file picker that leverages Android's Storage Access Framework (SAF). In Flutter, a common choice is the [file_picker](#) plugin [16†L130-L139]. Invoking something like `await FilePicker.pickFiles()` will open the native Android file picker UI. Critically, **no special permission is needed** for this: Flutter's file picker uses SAF, so when the user selects a file, the app receives a URI with one-time access. The Android docs explicitly note that *"Because the user is involved in selecting the files... this mechanism doesn't require any system permissions"* [32†L206-L214] [32†L217-L219]. In practice, ensure:

- You add the `file_picker` plugin and the recommended setup (AndroidManifest entries, see its docs).
- On Android 13+ declare the new media permissions (`READ_MEDIA_IMAGES` etc) **only** if you need to read media directly. But for `file_picker`, the user's explicit selection suffices (no `READ_EXTERNAL_STORAGE` needed) [32†L206-L214].
- For broad file access (rare in this app), you'd request `MANAGE_EXTERNAL_STORAGE` on Android 11+, but that's only for writing arbitrary files.

Once integrated, your code might look like:

```
FilePickerResult? result = await FilePicker.pickFiles(  
  type: FileType.image,  
  allowMultiple: false,  
);  
if (result != null) {  
  String path = result.files.single.path!;  
  // Load and use the image at `path`  
}
```

Make sure to handle user cancellation. Also test on real devices: on Android 13, ensure the manifest has `<uses-permission android:name="android.permission.READ_MEDIA_IMAGES"/>` if you plan to pick images. But remember: for SAF, the plugin's documentation notes that the user's selection *implicitly grants* access [32†L206-L214].

Design Tree & Editable Layers Panel

The app's **Design Tree** should be a hierarchical, collapsible view (like a layers panel in Photoshop or Figma) that reflects each garment and its sub-elements. For example:

- **Kurta** (expandable)
 - ☐ Base Fabric
 - ☐ Base Color
 - ☐ Hero Motif
 - ☐ Secondary Motif
 - ☐ Filler Motif

- ☐ Outline
- ☐ Sleeve Border
- ☐ Sleeve Color
- ☐ Neckline Panel
- ☐ Hem Border
- ☐ Shadow
- **Pant**
 - ☐ [similar layers for pant]
- **Dupatta**
 - ☐ [layers for pallu, body, borders, etc]

Each sub-item should have controls (checkbox or toggle icon, color swatch, import button, regenerate button, etc.) to include/exclude and edit that layer. This mirrors standard layer panels: “layers arranged in a stack in the layer panel... to help designers organize and design elements” [23†L59-L62] . Implement this as a collapsible tree view (Flutter packages like [animated tree view](#) or custom ExpansionTiles can do this). Follow UI best-practices for tree views: **lazy load or build children on expand** if the tree is large, **clear indentation or glyphs** for hierarchy, and a global “Expand/Collapse All” function [20†L169-L177] [20†L180-L183] .

Within each layer entry, provide actions: e.g. an “Import” icon to load an image or SVG for that layer, and an “AI” or “Regenerate” icon to run the AI on that layer only. For instance, if the user clicks the Hero Motif layer’s AI button, the orchestrator should treat the current motif area as the mask and regenerate it. Similarly, allow the user to import a specific border file or use AI to create one. In effect, any layer (like “Sleeve Border” or “Neck Motif”) can be independently locked, hidden, colored, imported or AI-edited. This gives maximal control: exactly the user-requested “import or AI modify each layer” workflow.

Reference: Tree views are “essential for file explorers [and] complex category navigation” [20†L169-L177] . Plan your layer tree accordingly and label layers clearly. (Good layer naming and grouping makes future UX and development smoother [23†L59-L62] .)

AI Chat Attachments & Asset Mapping

The AI assistant interface must allow *attaching assets* directly in the chat so the AI can reference them. A common pattern (seen in apps like WhatsApp or Slack) is a paperclip/“+” icon beside the text field. When clicked, it opens a file picker or asset browser. For our use-case, that menu could include “Gallery (Fabric)”, “Camera (Fabric)”, “Import Motif”, “Project Files”, etc. Once the user selects an image (e.g. a border or motif), show it as a small thumbnail chip inside the chat input box [27†L37-L43] [27†L47-L55] .

For example, Uno’s HotDesign AI agent has exactly this flow: the user clicks an attachment icon, picks one or more images, and the images appear as thumbnails (16px chips) in the chat input [27†L37-L43] [27†L47-L55] . When the user then types a prompt and sends, the AI receives both the text and the images. The images serve as visual references – e.g. “Use this motif only on the pallu.” As Uno docs explain, “The Hot Design Agent supports image attachment... allowing you to include images in your chat prompts” [27†L17-L20] .

In our app, do the same: implement a chat input with an attachment button. Use a `FilePicker` (or custom asset picker) to import the chosen asset. Display thumbnails of attached images in the input (users should be able to remove them before sending). When the message is sent, your Conversation Parser/AiOrchestrator will receive an **AIRequest** that includes (1) the current design state, (2) masks, (3) the textual prompt, and (4) references to any attached assets (with their URIs or image data). The AI logic can then incorporate the attached image – for instance, replacing the selected layer with the uploaded motif.

This follows best-practice for chat UIs: it's much like WhatsApp's attachment flow [25†L59-L63] , and is explicitly recommended for AI design assistants [27†L17-L20] [27†L37-L43] .

Prioritized Fix List & Developer Tasks

Based on the above, the top issues to address are:

- **Critical UI fixes:** Enforce safe-area padding so no content overlaps the status bar. Fix the landscape layout by using responsive widgets (e.g. `SafeArea` , `OrientationBuilder` , conditional layouts) [2†L62-L68] [5†L19-L23] .
- **Asset import functionality:** Integrate a file picker for images. Ensure the AndroidManifest and runtime code allow selecting gallery/files (no extra permission needed if using SAF) [32†L206-L214] . Test that after picking, the image can be loaded into the app. Add all required Android permissions (`READ_EXTERNAL_STORAGE` for <API30 if necessary, or `READ_MEDIA_IMAGES` for API33+) and handle granted/denied states.
- **Design Tree implementation:** Build a tree view (expandable list) for each category (Kurta, Pant, Dupatta, etc). For each layer, add controls (show/hide, lock, color swatch, import, AI-regenerate). Use clear indentation and give users "Expand All/Collapse All" options [20†L169-L177] [20†L180-L183] . This may involve creating a data model for layers and iterating it in UI.
- **AI Chat UI & Orchestrator integration:** Add an attachment (paperclip) icon to the chat input. Hook it up to the same file-picker or asset library. Display chosen images as thumbnails in the chat field [27†L37-L43] [27†L47-L55] . Modify the AI Orchestrator so it packages these attachments into the `AIRequest` and processes them: e.g. if a motif image is attached and the user says "apply this motif to the border", the orchestrator should apply the mask for the border and use the image in prompt construction.
- **AI Conversation logic:** Ensure the backend can parse instructions referring to layers (see "conversation parser" earlier). Integrate the image analysis so the AI knows *what* the uploaded asset is (e.g. this is a motif image). This may involve extending the ConversationContext to include last-attached asset metadata.

In code, this translates to tasks like:

- **UI Tasks:** Wrap your main scaffold in `SafeArea` . Use `MediaQuery` or `LayoutBuilder` to adjust padding in landscape. Refactor the layout to not rely on fixed heights; use `Expanded` / `Flexible` so the toolbar and tree panel are contained. Test on multiple devices/orientations.
- **File Picker:** Add `file_picker: ^11.x` to `pubspec.yaml` . In Android `manifest.xml` , add `<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>` (and `READ_MEDIA_IMAGES` for API 33+). Write Dart code to call `FilePicker.pickFiles()` . After

picking, load the image file into a widget (e.g. `Image.file(File(path))`) in your importer workflow. Ensure the picker dialog actually appears when “Import” is tapped.

- **Design Tree:** Create a `DesignTreeWidget`. Use `ExpansionTile` or a `TreeView` package. Each expandable parent (e.g. Kurta) shows a list of layers (`ListTile` with checkbox and icons). Maintain a state model (per [23]) so selections/locks persist. Provide methods to import or edit the specific layer when its action icon is pressed.
- **Chat Attachments:** In the chat screen widget, add an `IconButton` for attachments. On press, call the file picker for images. When files are picked, add them to the message draft state; show small `Image.memory()` or `Image.file` thumbnails inline with the `TextField`. Update the send button handler to include attachments (e.g. as byte arrays) in the AI request.
- **Backend Logic:** Update your `AIRequest` model to include a list of attached images (as base64 or file URIs). Modify the AI Orchestrator so it concatenates prompts like “Use the attached image as the new motif for the [layer]” depending on user instruction. Ensure the Validator checks consistency of attached asset types (e.g. image must match the layer type).

By following these steps in order (fix UI → enable file import → implement tree → enhance chat), you’ll rebuild the workflow as: **Upload/Import → Analyze (AI) → Design Tree edit → AI attach/edit → Export**. Each step should be thoroughly tested before moving on.

Sources: We followed Flutter guidelines on `SafeArea` to avoid overlays [2†L62-L68] , Android docs on system gestures and display cutouts [5†L19-L23] , Flutter responsive layout patterns [29†L1-L4] [29†L7-L10] , Android Storage Access Framework notes [32†L206-L214] [32†L217-L219] , design-best-practices for tree views [20†L169-L177] [20†L180-L183] [23†L59-L62] , and AI chat attachment patterns [25†L59-L63] [27†L37-L43] [27†L47-L55] . All cited passages informed the recommendations above.
